

PROJECT AURORA

A Real-World Azure Cloud Onboarding Project

End-to-End Capstone - Combining Every AZ-900 Learning Path 4 Guided Project

SIWES Cloud Computing Training - Computer Training Institute

1. Project Charter

1.1 Background

Bright Retail Ltd. is a growing Nigerian retail company preparing to launch a new product line. As part of its first move to the cloud, the IT team has been asked to deliver a small, well-governed Azure pilot in four weeks: a public landing page, a lightweight backend endpoint, secure onboarding for a new hire, secure external file sharing, and full cost, security, and monitoring guardrails around it all.

You are the engineer running this pilot. This document is both your project brief and your deployment runbook - by the end, you will have built one coherent, real Azure environment that touches every skill from Learning Path 4, not eight disconnected practice exercises.

1.2 Business Objectives

- Publish a public product landing page with no servers to manage
- Provide a simple backend status endpoint for future integrations
- Onboard a new support analyst with least-privilege access from day one
- Share sensitive reports with an external auditor securely and revocably
- Keep the pilot's spend visible and its deployments compliant with policy
- Give the operations team early warning of platform issues or accidental deletions
- Prove the whole environment can be redeployed or torn down through automation, not just manual clicks

1.3 Stakeholders

Role	Who	Interest
Project sponsor	Bright Retail IT Manager	Wants a working, secure pilot delivered in 4 weeks
Marketing	Product marketing lead	Needs the landing page live and easy to update
Support team	Ada Chukwu (new analyst)	Needs read-only visibility into pilot resources
Finance	Finance business partner	Needs cost visibility and spend guardrails
External party	Independent auditor	Needs temporary, secure access to monthly reports
You	Cloud engineer (trainee)	Delivers, documents, and demos the finished pilot

1.4 Success Criteria

- All 8 phases below are deployed inside a single, consistently tagged resource group
- The landing page and status API are live and reachable
- Ada's access is provably read-only
- The SAS-shared report can be revoked on demand and auto-expires via a lifecycle rule
- A budget and a location policy are both actively enforced
- Ops receives a real alert email during testing
- The entire pilot can be redeployed from a single CLI script

2. Architecture at a Glance

Everything in this project lives inside one resource group, built up phase by phase. Keep these names consistent - they are referenced throughout every phase below.

Component	Name used throughout this project	Purpose
Resource group	rg-brightretail-pilot	Contains every pilot resource
Storage account (site)	stbrightretailweb	Static website (\$web) + SAS-shared reports (partner-reports)
Storage account (functions)	stbrightretailfunc	Backing storage required by the Function App
Function App	func-brightretail-status	Serverless status API (Flex Consumption)
Application Insights	appi-brightretail-status	Monitors the Function App
Entra ID user	ada.chukwu	New support analyst
Entra ID group	sg-brightretail-support-readers	Holds Reader access for support staff
Budget	budget-brightretail-pilot	Cost alerting at 80% / 100%
Policy assignment	policy-allowed-locations	Blocks deployments outside approved regions
Action group	ag-brightretail-ops	Email notification channel for ops alerts
Alerts	alert-service-health, alert-activity-rg-delete	Platform incidents and resource-deletion alerts
Tagging standard (applied to every resource) project=aurora, environment=pilot, owner=<your name>, department=it. Consistent tags are what make the cost and compliance phases (5 and 6) actually work - apply them from Phase 1 onward, not retroactively.		

3. Four-Week Execution Plan

This plan assumes your remaining training time is structured the same way as the original course: 2 sessions per week, 2 hours per session - 8 sessions in total. Each session delivers one phase of Project Aurora.

Week	Session	Phase	Focus
Week 1	Session 1	Phase 1: Governance Foundations (Tags & Locks)	Kick off the project like a real engagement - resource group, tags, delete lock.
Week 1	Session 2	Phase 2: Static Website on Blob Storage	Publish the landing page; first live, demoable deliverable.
Week 2	Session 3	Phase 3: Serverless Status API (Azure Functions)	Introduce Cloud Shell and a second compute pattern (serverless).
Week 2	Session 4	Phase 4: Identity & RBAC Onboarding	Entra ID user/group creation and least-privilege access.
Week 3	Session 5	Phase 5: Secure File Sharing with SAS	Stored access policies, revocation, and lifecycle management.
Week 3	Session 6	Phase 6: Cost Guardrails (Budget & Policy)	Tie earlier phases together under a single cost/compliance lens.
Week 4	Session 7	Phase 7: Monitoring (Service Health & Activity Log)	Operational readiness - alerts and action groups.
Week 4	Session 8	Phase 8: Automation with Cloud Shell & CLI + Final Demo	Script the whole pilot, present the finished portfolio project.

Facilitator note

Each phase is self-contained but builds on resources from earlier phases (see each phase's Prerequisites). If a session runs short, the Portfolio Evidence step at the end of each phase can be finished as homework before the next session begins.

4. Detailed Phase-by-Phase Deployment Guide

Work through these phases in order - each one deploys into the same rg-brightretail-pilot resource group and several phases reuse resources created earlier.

Phase 1: Governance Foundations — Tags & Locks

Before any resource goes live, Bright Retail's IT policy requires every pilot resource group to be tagged for cost ownership and protected against accidental deletion. This is the first thing you do on any real project — not an afterthought.

Objective:

Create the pilot resource group, apply a consistent tagging standard, and lock it against accidental deletion.

Prerequisites:

- An active Azure subscription (free tier or pay-as-you-go)
- Owner or Contributor access on the subscription

Resources & naming used in this phase:

- Resource group: rg-brightretail-pilot
- Region: choose the region closest to you (e.g. South Africa North, or another region your subscription supports)
- Tags applied to the resource group: project=aurora, environment=pilot, owner=<your name>, department=it

Step-by-Step Deployment:

1. Sign in to portal.azure.com.
2. Search for "Resource groups" and click "Create".
3. Name it rg-brightretail-pilot, choose your region, and click "Next: Tags".
4. Add the four tags listed above (one key-value pair per row), then click "Review + create" and "Create".
5. Once created, open rg-brightretail-pilot and confirm the Tags blade shows all four tags.
6. In the resource group's left menu, click "Locks" > "Add".
7. Name the lock lock-prevent-delete, set Lock type = "Delete", add a note explaining why (e.g. "Pilot resource group — do not delete without IT sign-off"), and click "OK".
8. Try deleting the resource group to confirm Azure blocks the action because of the lock, then cancel the deletion.

Validation — how to know it worked:

The resource group exists, shows all four tags under its Tags blade, and attempting to delete it is blocked with a message referencing your delete lock.

Tips & Troubleshooting

Keep this tag set identical on every resource you add in later phases — consistency is what makes tag-based cost reports useful. • A delete lock does not block normal read/write operations, only deletion — you can still create resources inside it.

Portfolio evidence to capture

Screenshot of the resource group's Tags blade, and a screenshot of the blocked delete attempt with the lock's error message visible.

Phase 2: Publish the Product Landing Page

Marketing needs a new product landing page live before the next promotion. Bright Retail doesn't want to pay for a web server just to host a few static pages, so you deploy it directly from Blob Storage.

Objective:

Create a storage account, enable static website hosting, and publish the landing page.

Prerequisites:

- Phase 1 completed (rg-brightretail-pilot exists)

Resources & naming used in this phase:

- Storage account: stbrightretailweb (must be globally unique — add initials/numbers if taken)
- Static website container: \$web (created automatically by Azure)
- Files to upload: index.html (landing page) and error.html (404 page)

Step-by-Step Deployment:

1. In the portal, search "Storage accounts" and click "Create".
2. Select rg-brightretail-pilot as the resource group, name it stbrightretailweb, choose your region, and leave Performance = Standard, Redundancy = LRS (sufficient for a pilot).
3. On the Tags tab, apply the same four tags from Phase 1 plus workload=landing-page.
4. Click "Review + create", then "Create", and wait for deployment.
5. Open the storage account, click "Static website" (under Data management), and set it to Enabled.
6. Set Index document name to index.html and Error document path to error.html, then Save — copy the Primary endpoint URL shown.
7. Click "Containers" and open the automatically created \$web container.
8. Click Upload and add your index.html and error.html files (create simple ones on your computer beforehand, with Bright Retail branding/content).
9. Open the Primary endpoint URL in a browser to confirm the landing page is live.
10. Edit your index.html locally, change something visible (e.g. a headline), re-upload it to \$web, and refresh the browser to confirm the update appears immediately with no redeployment step.

Validation — how to know it worked:

Visiting the Primary endpoint URL shows your landing page, and a content update you upload appears live within seconds of refreshing.

Tips & Troubleshooting

Storage account names must be lowercase letters and numbers only, 3–24 characters. • If you get a 404, confirm both files sit directly inside \$web and are named exactly index.html / error.html.

Portfolio evidence to capture

Screenshot of the live site at its Primary endpoint URL, and a before/after screenshot showing your content update went live.

Phase 3: Add a Serverless Status API

The support team wants a lightweight endpoint the landing page (and future apps) can call to check that backend services are healthy — without Bright Retail managing a single server.

Objective:

Deploy an HTTP-triggered Azure Function on a Flex Consumption plan, secure it, and confirm monitoring is capturing its activity.

Prerequisites:

- Phase 1 completed (rg-brightretail-pilot exists)
- Azure Cloud Shell access (Bash)

Resources & naming used in this phase:

- Function App: func-brightretail-status
- Storage account for the function: stbrightretailfunc (functions require their own linked storage account)
- Function name: GetStatus
- Application Insights resource: appi-brightretail-status (created alongside the Function App)

Step-by-Step Deployment:

1. In the portal, search "Function App" and click "Create".
2. Select rg-brightretail-pilot, name it func-brightretail-status, choose "Flex Consumption" as the hosting plan, and pick your runtime stack (Node.js or Python) and region.
3. On the Storage tab, create a new storage account stbrightretailfunc.
4. On the Monitoring tab, confirm Application Insights is enabled and named appi-brightretail-status.
5. Apply the standard tags (project=aurora, environment=pilot, owner, department=it) on the Tags tab, then Review + create, then Create.
6. Once deployed, open the Cloud Shell (the >_ icon in the top toolbar) and select Bash.
7. In Cloud Shell, scaffold a simple HTTP-triggered function named GetStatus using the Azure Functions Core Tools (func init and func new), or use the portal's built-in code editor under Functions > Create > HTTP trigger to write a small function that returns a 200 OK with a JSON status message.
8. Deploy the function (via func azure functionapp publish func-brightretail-status from Cloud Shell, or Save directly if using the portal editor).
9. Open the function's "Get Function Url" option, copy the URL (it includes a function key), and open it in a browser to confirm you get a 200 OK JSON response.
10. Try calling the same URL without the function key query parameter and confirm you're denied — this proves the endpoint is secured, not anonymous.
11. In the Function App's left menu, click "Application Insights" > "View Application Insights data", then check the Live Metrics or Logs to confirm your test calls were recorded.

Validation — how to know it worked:

Calling the function URL with its key returns a 200 OK JSON response; calling it without the key is denied; and the calls are visible in Application Insights logs.

Tips & Troubleshooting

Flex Consumption bills only for execution time — perfect for a low-traffic status endpoint. • If Application Insights shows no data yet, wait 2-3 minutes; ingestion isn't instant.

Portfolio evidence to capture

Screenshot of the successful JSON response with the key, the denied response without it, and the Application Insights log entry for your test call.

Phase 4: Onboard the New Support Analyst

Bright Retail just hired a support analyst, Ada, who needs to view the pilot's resources to help troubleshoot issues — but should not be able to change or delete anything. This is a textbook least-privilege access request.

Objective:

Create a user and security group in Microsoft Entra ID, and assign a Reader role scoped to the resource group.

Prerequisites:

- Phase 1 completed (rg-brightretail-pilot exists)
- Global Administrator or User Administrator rights in Microsoft Entra ID

Resources & naming used in this phase:

- User: ada.chukwu@<yourtenant>.onmicrosoft.com, display name "Ada Chukwu"
- Security group: sg-brightretail-support-readers
- Role assignment: Reader, scoped to rg-brightretail-pilot

Step-by-Step Deployment:

1. In the portal, search "Microsoft Entra ID" and open it.
2. Click "Users" > "New user" > "Create new user". Set the username to ada.chukwu, display name "Ada Chukwu", and let Entra ID auto-generate a temporary password (or set one) — then Create.
3. Click "Groups" > "New group". Set group type = Security, group name = sg-brightretail-support-readers, and add Ada Chukwu as a member. Click Create.
4. Navigate to your resource group rg-brightretail-pilot, click "Access control (IAM)" in the left menu.
5. Click "Add" > "Add role assignment", search for and select the Reader role, click Next.
6. Under "Assign access to", choose "User, group, or service principal", click "Select members", search for sg-brightretail-support-readers, select it, and click Review + assign twice.
7. Sign out, then sign in as ada.chukwu (using an in-private/incognito browser window) with the temporary password, and set a permanent password when prompted.
8. As Ada, navigate to rg-brightretail-pilot and confirm you can view all resources (storage account, function app, etc.).
9. As Ada, try to create a new resource inside rg-brightretail-pilot (e.g. add a new storage account) and confirm you're denied due to insufficient permissions.
10. Sign back in as the administrator and, in Access control (IAM) > Role assignments, confirm the Reader role is listed against sg-brightretail-support-readers at the resource group scope.

Validation — how to know it worked:

Ada can view every resource in rg-brightretail-pilot but is denied when attempting to create, modify, or delete anything — proving Reader access is correctly scoped.

Tips & Troubleshooting

Always assign roles to groups, not individual users — when someone leaves or changes teams, you update group membership instead of hunting down every individual assignment. • RBAC is enforced by Azure Resource Manager, not inside the application — it controls management-plane access, not application-level data access.

Portfolio evidence to capture

Screenshot of the Reader role assignment on the group at resource group scope, and a screenshot of the access-denied message when Ada attempted to create a resource.

Phase 5: Share the Monthly Report with an External Auditor

Bright Retail's external auditor needs temporary access to this month's sales report — without an Azure account, without a permanent password, and without the file staying accessible forever.

Objective:

Upload a report to a private container, generate a time-boxed SAS link from a stored access policy, and confirm you can revoke it instantly.

Prerequisites:

- Phase 2 completed (stbrightretailweb storage account exists)

Resources & naming used in this phase:

- Container: partner-reports (private, created inside stbrightretailweb)
- File: monthly-report.pdf
- Stored access policy: policy-partner-30day
- Lifecycle management rule: rule-autodelete-reports (auto-deletes files after 30 days)

Step-by-Step Deployment:

1. In stbrightretailweb, click "Containers" > "+ Container". Name it partner-reports, set Public access level = Private, and click Create.
2. Open partner-reports, click Upload, and add a sample monthly-report.pdf (any short PDF or text file renamed for the exercise).
3. Try opening the file's URL directly in a private browser window and confirm you get an authentication error — this proves the container is not publicly accessible.
4. In the storage account's left menu, under Data storage > Containers, select partner-reports, then find "Access policy" (or "Shared access signature" depending on portal layout) and create a new stored access policy named policy-partner-30day with Read permission and a 30-day expiry window.
5. Generate a SAS token/URL for monthly-report.pdf using that stored access policy (rather than an ad-hoc SAS), and copy the full URL.
6. Open the SAS URL in a private browser window and confirm the file downloads successfully.
7. Go back to the stored access policy and delete it (or revoke it), then try the same SAS URL again and confirm access is now denied — proving instant revocation.
8. Recreate the stored access policy (or generate a fresh SAS) so the exercise resource is left in a working state for demonstration purposes.
9. In the storage account's left menu, click "Lifecycle management" > "+ Add a rule". Name it rule-autodelete-reports, scope it to blobs in partner-reports, and set an action to delete blobs more than 30 days old since last modification, then Add.

Validation — how to know it worked:

The report is downloadable only via the SAS URL (not directly), the SAS URL stops working immediately after the stored access policy is deleted, and a lifecycle rule exists to auto-delete old reports.

Tips & Troubleshooting

Stored access policies are the recommended way to issue SAS tokens in a real project — they let you revoke every token generated from that policy in one action, instead of tracking individual token expiries. • Lifecycle management rules run on a schedule (roughly once per day), so don't expect instant deletion the moment a blob crosses the age threshold.

Portfolio evidence to capture

Screenshot of the successful SAS download, the denied access after revoking the policy, and the configured lifecycle management rule.

Phase 6: Set Up Cost Guardrails for the Pilot

Before the pilot scales beyond a handful of resources, Bright Retail's finance team wants visibility into spend and a hard rule preventing resources from being deployed outside approved regions.

Objective:

Confirm consistent tagging is in place, create a budget with alert thresholds, and enforce an allowed-locations policy.

Prerequisites:

- Phases 1-4 completed (several resources now exist in rg-brightretail-pilot)

Resources & naming used in this phase:

- Budget: budget-brightretail-pilot (monthly, alerts at 80% and 100%)
- Policy assignment: policy-allowed-locations, scoped to rg-brightretail-pilot

Step-by-Step Deployment:

1. Open rg-brightretail-pilot, click "Tags", and confirm every resource you created in Phases 1-4 shares the same project=aurora, environment=pilot, owner, and department=it tags — add any that are missing.
2. Search "Cost Management + Billing" (or open Cost Management directly on the resource group), click "Budgets" > "+ Add".
3. Scope the budget to rg-brightretail-pilot, name it budget-brightretail-pilot, set a monthly amount appropriate to a pilot (e.g. equivalent to \$10-20), and set the reset period to Monthly.
4. Add two alert conditions: 80% of budget (Actual) and 100% of budget (Actual), entering your email address for both.
5. Click Create to save the budget.
6. Search "Policy" and open Azure Policy.
7. Click "Assignments" > "Assign policy".
8. Set Scope to rg-brightretail-pilot, click "Select a definition", search for and choose the built-in policy "Allowed locations", and click Select.
9. On the Parameters tab, choose only the region(s) you've been deploying to (e.g. your chosen region only).
10. Set the policy effect to Deny (this is usually the default for this definition), name the assignment policy-allowed-locations, and click Review + create, then Create.
11. Test enforcement: try creating a new storage account inside rg-brightretail-pilot in a different, non-allowed region, and confirm the deployment is blocked with a policy violation message.

Validation — how to know it worked:

The budget shows current spend against your set amount with two alert thresholds configured, and attempting to deploy a resource in a non-allowed region is blocked by the policy.

Tips & Troubleshooting

Policy evaluation can take a few minutes to propagate after assignment — if a test deployment isn't blocked immediately, wait and retry. • Budgets alert you; they don't stop spending on their own — the Allowed Locations policy is your hard control, the budget is your early warning.

Portfolio evidence to capture

Screenshot of the budget's configured thresholds, and a screenshot of the policy violation message when deploying outside the allowed region.

Phase 7: Set Up Operational Monitoring

The IT operations team needs to know immediately if Azure has a platform-level incident affecting Bright Retail's region, or if someone accidentally deletes something inside the pilot resource group.

Objective:

Create an action group, then wire up a Service Health alert and an Activity Log alert to notify the ops team by email.

Prerequisites:

- Phase 1 completed (rg-brightretail-pilot exists)

Resources & naming used in this phase:

- Action group: ag-brightretail-ops (short name: BRops)
- Service Health alert: alert-service-health
- Activity Log alert: alert-activity-rg-delete

Step-by-Step Deployment:

1. Search "Monitor" and open Azure Monitor.
2. Click "Alerts" > "Action groups" > "+ Create".
3. Select rg-brightretail-pilot as the resource group, name it ag-brightretail-ops, set the short name to BRops.
4. On the Notifications tab, add an Email notification type with your email address, name it ops-team-email, then Review + create, then Create.
5. Back in Azure Monitor, search "Service Health" and open it.
6. Click "Health alerts" > "+ Create service health alert".
7. Set Subscription to your subscription, Services to "All services" (or your relevant services), Regions to the region(s) you're using, and Event types to Service issues and Planned maintenance.
8. On the Actions tab, select your existing ag-brightretail-ops action group, name the alert rule alert-service-health, and click Create.
9. Search "Activity log" and open it, then click "Add activity log alert" (or go through Azure Monitor > Alerts > + Create > Alert rule and set the scope to rg-brightretail-pilot).
10. Set the scope to rg-brightretail-pilot, condition = Operation name "Delete Resource Group" (or "Delete" at the resource group level), and severity as appropriate.
11. On the Actions tab, select ag-brightretail-ops, name the alert alert-activity-rg-delete, and click Create.
12. Test the pipeline: perform a small, safe deletable action (such as deleting a test blob or a throwaway test resource you create just for this) inside rg-brightretail-pilot, and confirm you receive an email via the action group within a few minutes.

Validation — how to know it worked:

Both alert rules appear as Enabled under Azure Monitor > Alerts > Alert rules, and a real triggering event results in an email notification arriving through ag-brightretail-ops.

Tips & Troubleshooting

Service Health alerts cover Azure's own platform issues; Activity Log alerts cover actions your team takes (or accidentally takes) inside your subscription — you need both for full operational awareness. • If no email arrives, check spam/junk folders first, then confirm the action group's email address was verified.

Portfolio evidence to capture

Screenshot of both alert rules showing Enabled status, and the email notification received during your test.

Phase 8: Automate Everything with Cloud Shell & the Azure CLI

Manually clicking through the portal for every new pilot environment doesn't scale. As a final deliverable, you write a single script that can stand up (or tear down) the whole Aurora pilot from the command line — the way a real cloud engineer would.

Objective:

Recreate the core resources from Phases 1, 2, and 6 using Azure CLI commands in Cloud Shell, then tag, query, and clean up using CLI only.

Prerequisites:

- Completed Phases 1-7 (for reference — this phase re-creates a parallel, disposable copy to prove automation skills)
- Access to Azure Cloud Shell (Bash)

Resources & naming used in this phase:

- New resource group for this exercise: rg-brightretail-automation
- Storage account: stbrightretailauto (must be globally unique)
- All resources tagged: project=aurora, environment=automation-demo, owner=<your name>

Step-by-Step Deployment:

1. Open Cloud Shell from the portal toolbar (the >_ icon) and select Bash if prompted.
2. Confirm you're signed in to the right subscription: `az account show --output table`.
3. Create the resource group: `az group create --name rg-brightretail-automation --location <your-region> --tags project=aurora environment=automation-demo owner=<your-name>`.
4. Create a storage account inside it: `az storage account create --name stbrightretailauto --resource-group rg-brightretail-automation --location <your-region> --sku Standard_LRS --tags project=aurora environment=automation-demo`.
5. List every resource in the group as a table: `az resource list --resource-group rg-brightretail-automation --output table`.
6. Update a tag on the storage account without touching anything else: `az tag update --resource-id $(az storage account show --name stbrightretailauto --resource-group rg-brightretail-automation --query id -o tsv) --operation merge --tags reviewed=true`.
7. Query just the names and locations of resources with a JMESPath filter: `az resource list --resource-group rg-brightretail-automation --query "[].{Name:name, Location:location}" --output table`.
8. Create a simple budget from the CLI for this resource group (optional stretch goal) using `az consumption budget create`, referencing your resource group's resource ID as scope.
9. Save all the commands above into a single file named `deploy-aurora.sh` in Cloud Shell's storage (using `code deploy-aurora.sh` or `nano deploy-aurora.sh`), so the whole environment can be redeployed with one command in future.
10. When you're done demonstrating, clean up this exercise's resources: `az group delete --name rg-brightretail-automation --yes --no-wait`.
11. Confirm cleanup by re-running `az resource list --resource-group rg-brightretail-automation --output table` after a minute or two and observing the group no longer exists.

Validation — how to know it worked:

Every command runs without error, the resource list and query commands return correctly filtered output, the script file is saved in Cloud Shell, and the resource group is fully removed by the final delete command.

Tips & Troubleshooting

Cloud Shell's storage persists between sessions, so your `deploy-aurora.sh` script will still be there next time you open Cloud Shell.

- `--no-wait` on the delete command returns control to you immediately instead of blocking the terminal while Azure deletes resources in the background.

Portfolio evidence to capture

A copy of your final `deploy-aurora.sh` script, and a terminal screenshot showing the resource group successfully created, queried, and deleted.

5. Turning This Into a Portfolio Piece

A finished deployment is only half the value - the other half is being able to show and explain it. Use the checklist below to package Project Aurora for a CV, LinkedIn, or interview.

5.1 Build a project write-up

- Create a short README (Word doc, PDF, or a public GitHub repo's README.md) titled "Project Aurora - Azure Cloud Onboarding Pilot".
- Include: the business scenario (adapt the background from Section 1.1), the architecture summary table from Section 2, and one screenshot per phase from the Portfolio Evidence boxes.
- Add a short "What I learned" paragraph per phase - this is what interviewers actually ask about.
- Include your `deploy-aurora.sh` script from Phase 8 as an attached file - a script is stronger portfolio evidence than a screenshot alone.

5.2 Prepare a live or recorded demo

- Rehearse a 5-8 minute walkthrough: landing page live -> status API responding -> Ada's restricted access -> SAS link working then revoked -> budget/policy screen -> alert email -> CLI script running end-to-end.
- If presenting live isn't possible, record a short screen capture instead - this is a strong, reusable asset for interviews.

5.3 CV / LinkedIn bullet points you can use

- Designed and deployed a governed Azure environment (resource groups, tags, delete locks) for a simulated retail pilot project.
- Published a serverless static website on Azure Blob Storage and a secure HTTP-triggered Azure Function on a Flex Consumption plan.
- Implemented least-privilege access using Microsoft Entra ID groups and Azure RBAC, and secure, revocable file sharing using SAS tokens with stored access policies.
- Configured cost guardrails (budgets, tagging) and governance controls (Azure Policy) alongside Service Health and Activity Log alerting.
- Automated environment deployment and teardown using the Azure CLI in a single reusable Bash script.

6. Final Go-Live Checklist

- Resource group rg-brightretail-pilot exists, fully tagged, and delete-locked
- Landing page is live at its static website endpoint and was successfully updated once
- Status API returns 200 OK with its function key and is denied without it
- Ada Chukwu's account has Reader-only access, proven by a denied write attempt
- SAS link for the monthly report worked, then was proven revocable, and a lifecycle rule is configured
- Budget with 80%/100% alerts is active; Allowed Locations policy blocks non-compliant deployments
- Service Health and Activity Log alerts both fired a real test notification via the action group
- deploy-aurora.sh successfully creates, queries, and tears down a parallel environment
- Portfolio write-up and demo (live or recorded) are ready to present

Facilitator sign-off

Once every item above is checked, this project satisfies full coverage of AZ-900 Learning Path 4 and is ready to be presented as a completed capstone / portfolio piece.